

flex 弹性布局

通过将元素的 display 属性设置为 flex, 可以控制子元素的排列和布局

任何一个容器都可以指定为 flex 布局, 行内元素也可以使用 flex 布局

```
.box {  
  display: flex;  
}
```

```
.box {  
  display: inline-flex;  
}
```

使用 flex 布局后, 子元素的 "float" "clear" "vertical-align" 属性将失效

只对 (行内元素 (span, a, 文字)
行内块元素 (img, input, button)
表格单元格 (td, th)) 起效

行内替换元素		Vertical-align:	属性值	作用
块级替换元素		控制同一行内的元素	baseline (默认)	元素基线与文字基线对齐
input, textarea, select, img 这些替换元素也可以		→ (行内元素、行内块元素) 垂直方向上的对齐	top	元素顶部与当前行顶部对齐
			bottom	元素底部与当前行底部对齐
			middle	元素中点与文字基线 + 1/2 x-height 对齐
			... px	相对于基线向上/下偏移对应像素
		text-align:	属性值	作用
		控制块级元素内部	left (默认)	内容沿容器左侧对齐
		行内元素水平方向上的对齐	center	内容在容器水平居中对齐
			right	内容沿容器右侧对齐
			justify	内容两端对齐, 仅对多行文字生效

Flex 容器的子元素会自动具备 "行内块" 的特质 ☆

只有 Flex 容器的 "亲儿子" 会继承这个特性, 意思是 Flex 的作用范围只到下一代

Flex 容器-共有 6 大属性:

① flex-direction 内部元素的排列方式

row (默认) 子项从左至右排列

row-reverse 子项从右至左排列

column 子项从上至下排列

column-reverse 子项从下至上排列

② flex-wrap 子元素的换行方式(超出容器时)

nowrap (默认) 不换行, 压缩子项挤在一行

wrap 换行, 第一行在上方

wrap-reverse 反向换行, 第一行在下方

③ flex-flow 子元素的排列方式、换行方式的简写

是 ① ② 的简写

flex-flow: row wrap;

相当于

flex-direction: row;

flex-wrap: wrap;

④ align-content 设置多个元素组成的整块对齐方式

stretch 多行拉伸以填满容器高度
(需子项无固定高度)

(默认) flex-start 多行靠容器顶部对齐

flex-end 多行靠容器底部对齐

center 多行在容器垂直居中对齐

space-between 多行等距分布, 首尾两行贴近容器边缘

space-around 多行等距分布, 两侧间距相等, 首尾=中

space-evenly 多行等距分布, 包括首尾

关于交叉轴

→ 关于主轴

⑤ justify-content 子元素的水平对齐方式

flex-start (默认) 左

flex-end 右

center 居中

space-between 间隔相等, 左右贴近边缘

space-around 两侧间隔相等

space-evenly 项目之间、项目与容器边缘之间间隔相等

stretch 弹性项无固定主轴尺寸时,
拉伸填充容器主轴空间

⑥ align-items 子元素的垂直对齐方式

stretch (默认) 所有项目拉伸到容器高度

flex-start 所有项目顶部对齐

flex-end 所有项目底部对齐

center 所有项目垂直居中对齐

baseline 所有项目基线对齐

grid 网格布局

grid 布局将容器分为“行”“列”，产生单元格，然后指定项目所在单元格

生成块级网格：

```
.container {  
  display: grid;  
}
```

生成行级网格：

```
.container {  
  display: inline-grid;  
}
```

定义网格的行和列：

grid-template-rows 属性定义行, grid-template-columns 属性定义列

可以使用分数单位fr来代表网格容器中可用空间的一等份。 grid-template-rows: 100px 1fr 1fr;
grid-template-columns: 1fr 2fr 1fr;

定义网格的行间距和列间距：

row-gap 属性定义行间距, column-gap 属性定义列间距

其它：

⇒ repeat 函数

grid-template-rows: repeat(3, 200px); // 3行, 每行高200px

或 repeat(4, 1fr); // 4行

或 repeat(2, 1fr 2fr); // 1fr, 2fr, 1fr, 2fr

⇒ minmax 函数

grid-template-rows: minmax(100px, 400px); // 行高限制在100px~400px间
或 minmax(200px, 1fr);

⇒ auto-fill 关键字

自动填充可用空间, 创建尽可能多的行。

grid-template-rows: repeat(auto-fill, minmax(200px, 1fr));

例. 原粉色, 鼠标悬停时变为绿色

```

<style>
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  text-decoration: none;
  list-style: none;
}
.nav {
  width: 640px; height: 50px;
  margin: 20px auto;
  background-color: pink;
}
.nav ul {
  display: flex;
  width: 100%; height: 100%;
  justify-content: space-between;
  align-items: center;
}
.nav ul li {
  width: 80px; height: 50px;
  text-align: center;
}
.nav ul li a {
  display: block;
  height: 50px;
  line-height: 50px;
  font-size: 16px; color: white;
}
.nav a: hover {
  background-color: green;
  cursor: pointer; // 手形
}

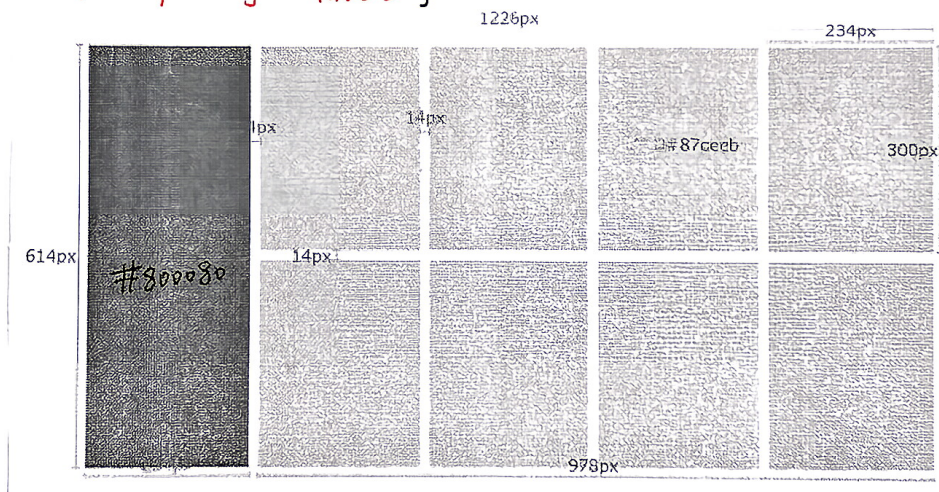
```

```

<body>
<div class="nav">
  <ul>
    <li><a href="#">新闻1</a></li>
    ... x 6
    <li><a href="#">新闻8</a></li>
  </ul>
</div></body></html>

```

例. 我的方法非常好, flex 与 grid 相结合



CSS/商品页.css:

```

* {margin: 0; padding: 0; box-sizing: border-box;}
.container {
  display: flex;
  justify-content: space-between;
  margin: 50px auto;
  width: 1226px; height: 614px;
}
.left {
  width: 234px; height: 614px;
  background-color: #800080;
}
.right {
  display: grid;
  grid-template-columns: repeat(4, 234px); // 4列, 每列宽度234px
  grid-template-rows: 300px 300px; // 2行, 每行高度300px
  column-gap: 14px; // 列间距
  row-gap: 14px; // 行间距
  width: 978px; height: 614px;
}
.item {
  width: 234px; height: 300px; background-color: #87ceeb;
}

```

商品页.html:

```

<!DOCTYPE html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>商品页</title>
  <link rel="stylesheet" href="css/商品页.css">
</head>
<body>
  <div class="container">
    <div class="left"></div>
    <div class="right">
      <div class="item"></div>
      ... x 6
      <div class="item"></div>
    </div>
  </div></body></html>

```

定位

用于解决盒子与盒子间的层叠问题或让盒子始终固定在屏幕中某个位置

设置定位有2种方法:

① 设置定位方式

position 属性 值

static (默认)	静态定位
relative	相对定位
absolute	绝对定位
fixed	固定定位

② 设置偏移量

应设置2个方向: 水平方向和垂直方向

依据就近原则, 水平和垂直各选一个方向去使用

方向	属性	属性值	含义
水平	left	数	距左边距离
水平	right	字	距右边距离
垂直	top	+	距上边距离
垂直	bottom	px	距下边距离

绝对定位 position: absolute

"拼爹型"定位, 相对于非静态定位的父元素进行移动(需配合方位属性)

会一直向外层找直至找到非static定位的父元素当参考物, 如果都没有就默认以浏览器的可视区域为参考来移动。

脱离标准文档流 (在页面中不占位置了, 原来位置被其他元素占据, 自己可能覆盖其他元素)

相对定位 position: relative

"自恋型"定位, 相对于自己之前的位置进行移动(需配合方位属性)

用于小范围内的移动

不脱标。

★ "子绝父相"找的是最近的有定位的父级

固定定位 position: fixed

"死心眼"型定位, 相对于浏览器的可视化区域进行定位移动(需配合方位属性)

用于让盒子固定在某位置

脱离标准文档流

css/lesson4.css :

```
* {margin:0; padding:0; box-sizing: border-box;}
```

```
.box {
  position: relative;
  width: 227px;
  height: 269px;
}
```

要给外面盒子设置宽/高, 否则"小hot"
图片按浏览器位置偏移

```
.hot {
  position: absolute;
  top: 5px;
  right: -3px;
}
```

方法一: 用标签 不用设置宽/高

方法二: 用background或background-image属性

```
* {margin:0; padding:0; box-sizing: border-box;}
```

```
.father {
  position: relative;
  width: 227px;
  height: 269px;
  background: url("../img/1.jpg");
}
```

```
.son {
  position: absolute;
  top: 5px; right: -3px;
  width: 40px; height: 24px;
  background-image: url("../img/2.png");
}
```

必须设置宽高(设置盒子的大小).
否则显示不出来

```
<body>
  <div class="father">
    <div class="son">
    </div>
  </div>
</body>
```

图片加载不成功时显示
替代文字

```
<div class="box">
  
  
</div>
</body>
</html>
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<style>
```

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

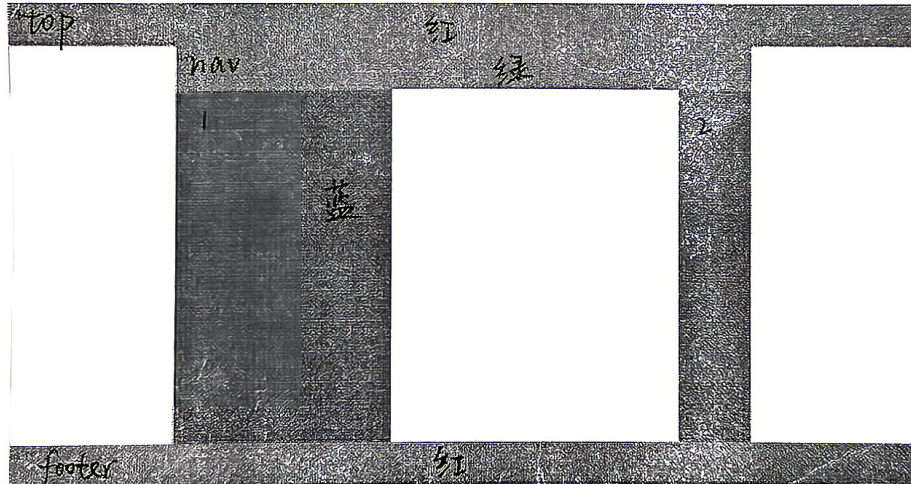
```
.top, .footer {
  background-color: red;
  height: 50px;
}
```

```
.nav {
  background-color: green;
  height: 60px;
  width: 60%;
  margin: 0 auto;
}
```

```
.content {
  display: flex;
  width: 60%;
  gap: 40px;
  margin: 0 auto;
}
```

```
</style>
```

```
</head>
```



```
.col1-left {
  background-color: blue;
  flex: 3;
  height: 400px;
}
```

```
.col2-right {
  background-color: blue;
  flex: 1;
  height: 400px;
}
```

```
<body>
```

```
<div class="top"> top </div>
<div class="nav"> nav </div>
<div class="content">
  <div class="col1-left"> 1 </div>
  <div class="col2-right"> 2 </div>
</div>
<div class="footer"> footer </div>
</body>
```

```
</html>
```

其实, h5 新增了一些语义化的标签

header - 页眉, nav - 导航, main(仅一个) - 内容, article - 独立完整的内容块,
section - 文档中的一个区块或章节, aside - 侧边栏、广告, footer - 页脚



```
<!DOCTYPE html>
<html lang="zh-CN">
```

```
<head>
```

```
<meta charset="UTF-8">
<title>商品展示</title>
<style>
  body {
    background-color: #f7f7f7;
  }
  .main {
    width: 1090px; height: 360px;
    border: 4px solid green;
    margin: 20px auto;
    position: relative;
    overflow: hidden;
  }
```

```
.main .nf {
  position: absolute;
  top: -20px; left: -50px;
}
.main .right {
  position: absolute;
  right: 10px; top: 10px;
  width: 497px; height: 334px;
  border: 2px solid green;
}
.main .time {
  position: absolute;
  left: 134px; top: 2px;
  width: 240px; height: 26px;
  background-color: black;
  color: white;
  text-align: center;
  line-height: 26px; ★
}
```

```
.main .des {
  display: block; text-decoration: none;
  position: absolute;
  left: 20px; top: 60px;
  font-size: 16px; line-height: 24px;
  color: #767676;
  width: 440px;
}
```

```
<body>
```

```
<div class="main">
```

```

```

```
<div class="right">
```

```
<p class="time">时间</p>
```

```
<a href="" class="des">介绍</a>
```

```

```

```

```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
```

```
.p1 {
```

```
position: absolute;
```

```
bottom: 15px;
```

```
left: 30px;
```

```
}
```

```
</style>
```

```
</head>
```

```
.p2 {
```

```
position: absolute;
```

```
bottom: 15px;
```

```
right: 30px;
```

```
}
```